

Comunicación Serial Básica Arduino

¿Qué es?

La comunicación serial es un método para transmitir datos entre el Arduino y el ordenador (o entre Arduino y otros dispositivos) a través del puerto USB. Permite que tu microcontrolador "hable" con el PC, enviando información que puedes ver en una consola. Arduino utiliza el protocolo **UART (Universal Asynchronous Receiver/Transmitter)** para esta comunicación.

¿Para qué sirve?

- Enviar mensajes desde Arduino al PC para ver el estado del programa
- Depurar tu código viendo qué sucede en cada momento
- Recibir instrucciones desde el PC hacia Arduino
- Monitorear sensores y valores en tiempo real

Baudios

Los baudios representan la velocidad a la que se transmiten los datos. Es como la velocidad de comunicación entre Arduino y tu PC. Los valores más comunes son:

- 9600: El más común, usado en la mayoría de proyectos básicos
- 115200: Más rápido, usado en proyectos más avanzados
- 19200, 38400, 57600: Velocidades intermedias

Para que Arduino y tu PC se entiendan, ambos deben usar la misma velocidad de comunicación.

Inicialización de la comunicación Serial

Para comenzar a usar la comunicación serial, debes inicializarla en la función `setup()` usando `Serial.begin()`.

```
Serial.begin(baudrate);
```

Ejemplo:

```
void setup() {  
  Serial.begin(9600); // Inicializa comunicación serial a 9600 baudios  
}
```

Enviar datos al PC desde el microcontrolador

Existen dos funciones principales para enviar datos:

`Serial.print()` : Envía datos a través del puerto serial **sin agregar un salto de línea** al final. Útil cuando quieres controlar exactamente cómo se formatean los datos.

Sintaxis:

```
Serial.print(data);
```

Ejemplo:

```
Serial.print("Hola");  
Serial.print(" ");  
Serial.print("Mundo");
```

// Resultado en consola: Hola Mundo

`Serial.println()`: Envía datos a través del puerto serial y **agrega un salto de línea** al final. Es más cómodo para cuando quieres cada dato en una línea diferente.

Sintaxis:

```
Serial.println(data);
```

Ejemplo:

```
Serial.println("Línea 1");
Serial.println("Línea 2");
Serial.println("Línea 3");
// Resultado en consola:
// Línea 1
// Línea 2
// Línea 3
```\n`
```

## ## Monitor Serie del IDE Arduino

El **Monitor Serie** es la herramienta que utiliza para ver los mensajes que envía tu Arduino.

**\*\*Cómo abrir el Monitor Serie:\*\***

1. En el IDE de Arduino, ve a: `Herramientas → Monitor Serie`
2. O usa el atajo: `Ctrl + Shift + M`

**\*\*Importante:\*\***

Asegúrate de que la velocidad del Monitor Serie coincida con la velocidad configurada en tu código (`Serial.begin()`). Si usas 9600 en tu código, debes seleccionar 9600 en el Monitor Serie.

## Tipos de datos que puedes enviar

Con `Serial.print()` y `Serial.println()` puedes enviar diferentes tipos de datos:

```
Serial.println(42); // Número entero
Serial.println(3.14); // Número decimal
Serial.println('A'); // Un carácter
Serial.println("Hola, mundo!"); // Una cadena de texto
Serial.println(true); // Un valor booleano (imprime 1 para true, 0 para false)
```

## ## Ejemplo completo básico

```
void setup() {
 Serial.begin(9600); // Inicializa comunicación serial
 Serial.println("¡Arduino iniciado!");
}
void loop() {
 Serial.println("Arduino está funcionando");
 delay(1000); // Espera 1 segundo antes de repetir
}
```

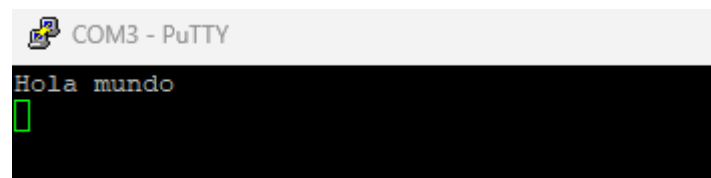
En este caso, como el texto solo queremos imprimirlo 1 sola vez, lo mejor sería ponerlo solo en el `setup()`:

```

P01_HolaMundo.ino
1 /*
2 programa 1: Hola mundo arduino
3 */
4
5 void setup() {
6
7 Serial.begin(9600);
8 Serial.println("Hola mundo");
9 }
10
11 void loop() {
12
13 }
14

```

Salida por consola:



```

COM3 - PuTTY
Hola mundo

```

## Puntos clave a recordar

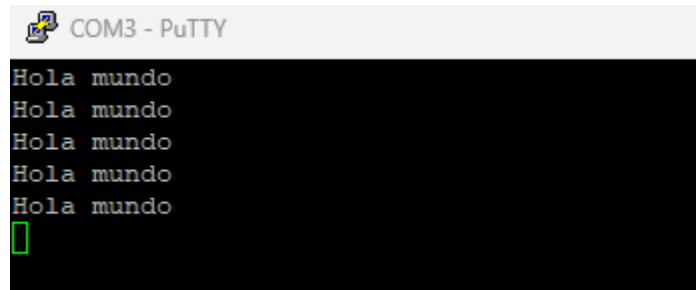
- **\*\*Siempre inicializa\*\*** Serial en `setup()` con `Serial.begin()`
- **\*\*Asegúrate\*\*** de que la velocidad del Monitor Serie coincida con tu código
- **\*\*Usa `println()`\*\*** cuando quieras cada dato en una línea nueva
- **\*\*Usa `print()`\*\*** cuando quieras controlar el formato exacto
- La comunicación serial es tu mejor amiga para depurar y entender qué hace tu código

En el 2º programa P02, se muestra un mensaje en bucle con un retardo de 1 segundo entre mensaje y mensaje:

```

P02_Mensaje_repetitivo.ino
1 ✓ /*
2 programa 2: Mensaje que se repiite cada 1 segundo
3 */
4
5 ✓ void setup() {
6
7 Serial.begin(9600); // Comunicación serial para visualizar resultado por consola
8
9 }
10
11 ✓ void loop() {
12 Serial.println("Hola mundo"); // mostrar texto por pantalla
13 delay(1000); // retardo de 1000 ms entre texto y texto
14 }
15

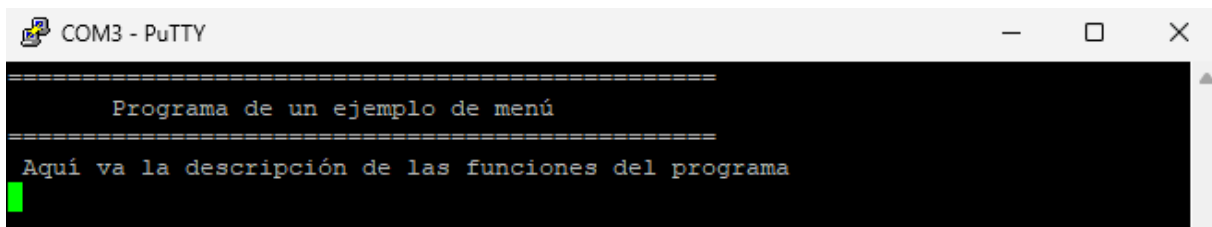
```



## Menú de consola

En los programas de consola es habitual que se muestre un menú para indicarle al usuario qué opciones tiene a la hora de interactuar con el programa. Como solo vamos a visualizar un ejemplo sencillo no funcional, lo podemos hacer perfectamente en el `setup()`:

```
P03_Multiples_mensajes_setup.ino
1 /*
2 programa 3: Múltiples mensajes (setup)
3 Sirve por ejemplo para crear un menú estático del programa de consola
4 */
5
6 void setup() {
7
8 Serial.begin(9600); // Comunicación serial para visualizar resultado por consola
9
10 // Primera línea
11 Serial.println("=====");
12 Serial.println(" Programa de un ejemplo de menú ");
13 Serial.println("=====");
14
15 Serial.println(" Aquí va la descripción de las funciones del programa ");
16 }
17
18 void loop() {
19 }
```



## Diferencias entre print() y println()

Con print() se muestran mensajes sin salto de línea, si se pone otro print(), el 2º mensaje se muestra a continuación del primero, con println() se incluye un salto de línea al final del texto:

```
P04_Diferencias_entre_print_y_println.ino
1 /*
2 programa 4: Diferencias entre print() y println()
3 Sirve por ejemplo para crear un menú estático del programa de consola
4 */
5
6 void setup() {
7
8 Serial.begin(9600); // Comunicación serial para visualizar resultado por consola
9
10 Serial.print("Este es el inicio de un mensaje con print("") "); Serial.print("Este es el final");
11 Serial.println("");
12 Serial.println("Este es el comienzo de un mensaje con println("") ");Serial.println("Este es el final");
13 }
14
15 void loop() {
16 }
```

```
COM3 - PuTTY
Este es el inicio de un mensaje con print() Este es el final
Este es el comienzo de un mensaje con println()
Este es el final
█
```

# Entrada de Datos por Comunicación Serial en Arduino

## ¿Por qué necesitamos entrada de datos?

Hasta ahora solo hemos **enviado datos** desde Arduino al PC. Pero Arduino también puede **recibir datos** desde el PC. Esto permite:

- Que el usuario controle el programa desde el Monitor Serie
- Cambiar el comportamiento en tiempo real
- Crear programas interactivos
- Leer comandos o valores

## Funciones de lectura serial

### Serial.available()

Devuelve **cuántos bytes están disponibles para leer** del puerto serial.

**Sintaxis:**

```
int bytesDisponibles = Serial.available();
```

**Ejemplo:**

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 Serial.println("Hay datos disponibles");
 }
}
```

**Nota:** Si no hay datos, devuelve 0. Es útil para verificar si hay algo que leer.

### Serial.read()

Lee **un byte** del puerto serial y lo devuelve como un número (0-255).

**Sintaxis:**

```
int byte = Serial.read();
```

**Importante:** `Serial.read()` elimina el byte leído del buffer, por lo que solo puedes leerlo una vez.

**Ejemplo:**

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 char character = Serial.read();
 Serial.print("Recibido: ");
 }
}
```

```
 Serial.println(caracter);
 }
}
```

## Serial.readString()

Lee una **cadena completa de caracteres** hasta encontrar un salto de línea (\n).

### Sintaxis:

```
String texto = Serial.readString();
```

### Ejemplo:

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 String mensaje = Serial.readString();
 Serial.print("Recibido: ");
 Serial.println(mensaje);
 }
}
```

**Nota:** Serial.readString() espera hasta recibir un salto de línea. En el Monitor Serie, presionar Enter envía \n.

## Serial.readStringUntil(char)

Lee caracteres **hasta encontrar un carácter específico**.

### Sintaxis:

```
String texto = Serial.readStringUntil(caracter);
```

### Ejemplo:

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 String numero = Serial.readStringUntil('\n');
 Serial.print("Recibido: ");
 Serial.println(numero);
 }
}
```

## Serial.parseInt()

Lee caracteres y los convierte a un **número entero**.

### Sintaxis:

```
int numero = Serial.parseInt();
```

### Ejemplo:

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 int numero = Serial.parseInt();
 Serial.print("Número recibido: ");
 Serial.println(numero);
 }
}
```

## Serial.parseFloat()

Lee caracteres y los convierte a un **número decimal**.

### Sintaxis:

```
float numero = Serial.parseFloat();
```

### Ejemplo:

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 float numero = Serial.parseFloat();
 Serial.print("Número decimal recibido: ");
 Serial.println(numero);
 }
}
```

## Patrón común: Leer y procesar datos

El patrón más usado es:

```
void loop() {
 // 1. Verificar si hay datos disponibles
 if (Serial.available() > 0) {

 // 2. Leer los datos
 String dato = Serial.readString();

 // 3. Procesar los datos
 Serial.print("Recibido: ");
 Serial.println(dato);
 }
}
```

```
}
}
```

## Cómo usar el Monitor Serie para enviar datos

- 1º Abre el Monitor Serie (Ctrl + Shift + M)
- 2º Escribe en el campo de texto en la parte superior
- 3º Presiona Enter o haz clic en "Enviar"
- 4º Arduino recibe los datos

## Tipos de datos al leer

Cuando recibes datos por serial, generalmente llegan como **texto (String)**, no como números. Por eso necesitas convertirlos:

- Serial.readString() → texto
- Serial.parseInt() → número entero
- Serial.parseFloat() → número decimal
- Serial.read() → un carácter individual

### Ejemplo de conversión:

```
void setup() {
 Serial.begin(9600);
}

void loop() {
 if (Serial.available() > 0) {
 // El usuario escribe "42" en el Monitor Serie
 String textoRecibido = Serial.readString(); // Obtiene "42" como
 texto

 // Convertir a número
 int numero = textoRecibido.toInt(); // Convierte "42" a 42

 Serial.print("Número: ");
 Serial.println(numero);
 }
}
```

## Diferencia entre read() y readString()

Función	Qué recibe	Qué devuelve
Serial.read()	Un carácter	Un byte (0-255)
Serial.readString()	Múltiples caracteres	Una cadena de texto
Serial.parseInt()	Dígitos	Un número entero
Serial.parseFloat()	Números con punto	Un número decimal

### Ejemplo:

```
// Si el usuario escribe "ABC" en el Monitor Serie:

char c = Serial.read(); // Obtiene 'A' (primer carácter)
String s = Serial.readString(); // Obtiene "ABC" (toda la cadena)
```

## Buffer serial

Arduino tiene un **buffer (memoria temporal)** de entrada de 64 bytes. Esto significa:

- Si envías más de 64 caracteres de una vez, algunos se pierden
- `Serial.available()` devuelve cuántos caracteres están en el buffer
- `Serial.read()` saca un carácter del buffer
- `Serial.readString()` vacía todo el buffer

### Ejemplo:

```
void setup() {
 Serial.begin(9600);
 Serial.println("Esperando datos...");
}

void loop() {
 if (Serial.available() > 0) {
 // Si hay datos, leerlos todos
 while (Serial.available() > 0) {
 char c = Serial.read();
 Serial.print(c);
 }
 Serial.println();
 }
}
```

## Puntos clave a recordar

- 1º **Siempre verifica** `Serial.available() > 0` antes de leer
- 2º **`Serial.read()`** devuelve un byte (carácter individual)
- 3º **`Serial.readString()`** devuelve una cadena completa
- 4º **`Serial.parseInt()`** convierte texto a número entero
- 5º **`Serial.parseFloat()`** convierte texto a número decimal
- 6º Los datos llegan como **texto**, no como números
- 7º El buffer tiene **64 bytes** de capacidad
- 8º **Presionar Enter** en el Monitor Serie envía un salto de línea (`\n`)

## Ejemplo lectura de 1 carácter

```
void setup() {

 Serial.begin(9600); // Comunicación serial para visualizar resultado
 por consola

 Serial.println(" ---- LECTURA DE DATOS -----");
 Serial.println("");
 Serial.println("Escribe algo y presiona Enter: ");
}

void loop() {

 // Verificar si hay datos disponibles
 if (Serial.available() > 0) {

 // leer un carácter
```

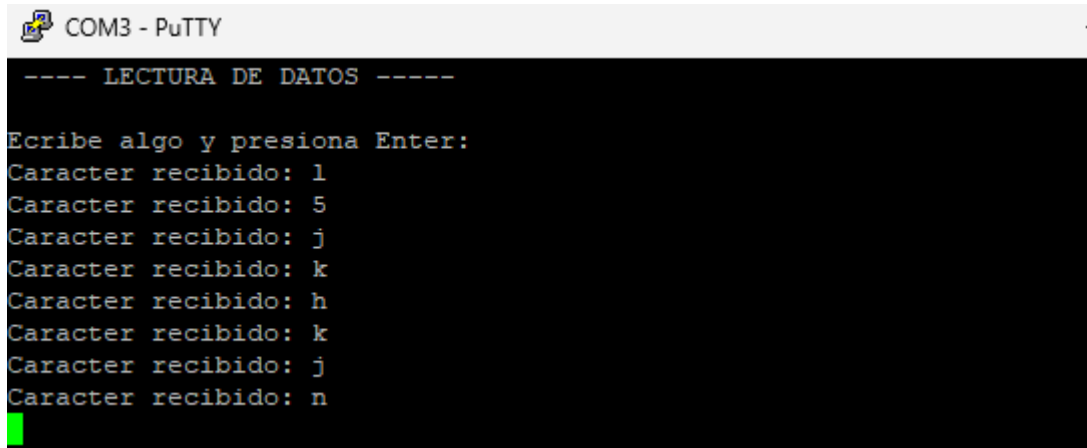
```

 char character = Serial.read();

 // Mostar el caracter introducido
 Serial.print("Caracter recibido: "); Serial.println(character);
 }
}

```

Salida:



```

COM3 - PuTTY
---- LECTURA DE DATOS ----
Escribe algo y presiona Enter:
Caracter recibido: l
Caracter recibido: 5
Caracter recibido: j
Caracter recibido: k
Caracter recibido: h
Caracter recibido: k
Caracter recibido: j
Caracter recibido: n

```

## Ejemplo completo

```

void setup() {
 Serial.begin(9600);
 Serial.println("=== LECTURA DE DATOS SERIAL ===");
 Serial.println("Escribe tu nombre y presiona Enter:");
}

void loop() {
 if (Serial.available() > 0) {
 // Leer lo que el usuario escribió
 String nombre = Serial.readString();

 // Procesar y mostrar
 Serial.print(";Hola, ");
 Serial.print(nombre);
 Serial.println("!");

 Serial.println("Escribe tu nombre y presiona Enter:");
 }
}

```